1025040904 Shuying Duan

1st Shuying Duan
School of Computer Science
Nanjing University of Posts and Telecommunications
Nanjing, China
1025040904@njupt.edu.cn

2nd Shuying Duan
School of Computer Science
Nanjing University of Posts and Telecommunications
Nanjing, China
1025040904@njupt.edu.cn

***Abstract*—The outbreak of viruses and continuous advancements in positioning systems have prompted researchers to propose a novel spatiotemporal query method known as the contact tracing query. Existing algorithms have already addressed both single-source and multi-source problems, but their judgment conditions are overly strict and thus not well adapted to real-world scenarios. This paper investigates the multi-source loose contact event query problem, a more flexible and practical problem. We first define the concept of multi-source loose contact events and propose a baseline query processing framework based on the idea of sliding window. To improve the query efficiency, we design an InGrid-MBR-based inverted index employed in the optimized processing(MLCEQ+). Comprehensive experiments on real datasets show that proposed query processing can effectively identify more potential contact events than existing algorithm, and MLCEQ+ can improve efficiency while guaranteeing accuracy.**

***Index Terms*—Moving Objects, Trajectory Data Mining, Contact Event.**

I. Introduction

With the rapid advancement of information technology, the widespread adoption of geographic information systems (GIS), mobile devices, the Internet of Things (IoT) and social networks has generated massive amounts of spatio-temporal trajectory data with moving objects. Spatio-temporal trajectory data not only capture the positional information of objects in space but also record the evolution of these positions over time, exhibiting significant multidimensional and dynamic characteristics. Such trajectories offer us information to understand moving objects and locations, enabling various applications in location-based social networks [1]. The analysis of trajectory data plays a critical role in diverse domains, ranging from personalized route recommendation systems [2] and real-time traffic congestion mitigation [3] to spatiotemporal behavior models of animals [4] and urban spatial computing frameworks [5]. As evidenced by previous studies, trajectory data pattern mining for moving objects holds both significant research value and promising application prospects.

Since the COVID-19 outbreak in 2019, the potential of trajectory data in modeling virus transmission has attracted increasing attention. However, traditional behavior models such as convoy [6], traveling companion [7], and trajectory clustering [8] are insufficient for capturing detailed contact paths. In response, researchers have proposed a novel spatio-temporal query termed the Contact Tracing Query (CTQ). They formulated the contact model and proposed a series of optimized query algorithms. Despite these advances, current CTQ methodologies exhibit strict limitations. For example, in Fig. 1, object o_2 maintains continuous interactions with multiple sources(s_1 and s_2) over time, aligning with traditional definition of sustained exposure leading to contact. In contrast, object o_1 exhibits non-continuous interactions with sources. Despite lacking prolonged continuous contact periods, o_1 might still be contacted if interactions meet critical risk thresholds. However, traditional CTQ approaches are incapable of detecting such loosely exposed objects. In real-world scenarios, once infected, object o_1 may subsequently interact with and infect other moving objects, initiating further chains of transmission. If these potential carriers are not effectively identified and isolated in time, they may contribute to undetected community spread, significantly undermining the effectiveness of epidemic control strategies. This highlights the urgent need for more flexible and comprehensive contact models capable of identifying non-continuous, high-risk exposures in multi-source environments.

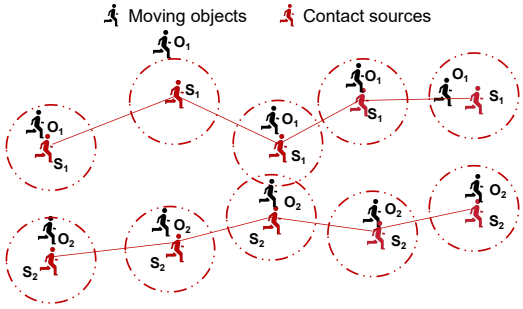


Fig. 1: An example of contact events

In this paper, we address the limitations of existing contact model by proposing multi-source loose contact. We first give the definition of the multi-source loose contact. Then we propose a baseline query algorithm based on sliding window(MLCEQ) to identify all potential contact events. To optimize efficiency, we further design a optimized query processing based on InGrid-MBR-based inverted index(MLCEQ+) to reduce redundant computations. Experiments on two real-world trajectory datasets demonstrate that our approach can

discover more potential contact events than the traditional algorithm. Overall, the contribution of this paper is given as follows.

- This paper introduces a novel definition of multi-source loose contact events and proposes a sliding-window-based baseline query processing framework, addressing the limitations of traditional contact tracing methods in real-world scenarios.
- To enhance query efficiency, we design an InGrid-MBR-based inverted index to filter out irrelevant objects, and further propose an optimized algorithm (MLCEQ+) to accelerate contact event detection.
- We performed an in-depth theoretical investigation and carried out comprehensive experiments on real-world datasets to assess the effectiveness of our proposed schemes.

II. Related Work

Contact tracing query (CTQ) has emerged as a critical tool to track virus transmission, particularly during the COVID-19 pandemic. The most prevalent implementations of contact tracing queries (CTQ) rely on mobile phone applications [9]–[15]. For example, TraceTogether [10], launched by the Singapore government, can track the proximity of two app users via Bluetooth. Numerous mobile app-based contact tracing methods have been developed, yet all such approaches remain constrained by hardware.

In recent years, contact tracing based on trajectory data has also been widely studied. Xu, et al. [16] define a contact event as occurring only when a moving object o remains within a distance d of a contact source s for a continuous, specified time period. Based on this definition, they implemented IMO, a toolbox for precise retrieval of contact events. In [17], Alarabi et al. present TraceAll—a real-time indoor contact-tracing approach that combines efficient spatial indices with optimized distance computations to ensure accurate tracing. These contact tracing models operate under a single-source assumption, where transmission chains are traced from a single source. To address the complexity of real-world multi-source contact events, researchers have developed multi-source contact tracing models. Chen et al. [18] introduced the concept of multi-source infection events (MSIE) and proposed the MIPM and MIPM+ algorithm for mining infection patterns in moving objects. Building on these work, Li et al. proposed two query optimization algorithms in 2023 [19] and 2024 [20]. In 2023, they improved efficiency using a 2D bitmap filter and anchor time scanning in the MCEQ+ algorithm. In 2024, they introduced ECEQ, which integrated a grid-time-object inverted index for even greater efficiency. These advancements enhance the performance of multi-source contact event queries. Table I presents some representative contact event methods from the aforementioned studies. Existing methods mainly check if moving objects have continuous contact with sources over a set time. However, in practical settings, such uninterrupted contact is often unrealistic. High-frequency but intermittent exposures within short durations may still lead to infection, yet remain undetected by these traditional models. As a result, current methodologies are insufficient to capture potential contact events arising from non-contiguous interactions, especially in multi-source environments.

To address this limitation, we present a mode definition for multi-source loose contact events and propose a novel query model for multi-source loose contact events, which relaxes the continuity constraint and better reflects real-world exposure risks. Furthermore, our method captures loose contact patterns from multiple sources using a sliding-window approach and an optimized inverted index for efficient, accurate query processing.

TABLE I: Overview of existing contact event query processing

Schemes	Single-source	Multi-source	loose contact event
IMO [16]	√	×	×
TraceAll [17]	√	×	×
MSIE+ [18]	√	√	×
MCEQ+ [19]	√	√	×
ECEQ [20]	√	√	×
MLCEQ+(This paper)	√	√	√

III. Problem Formulation

In this section, we give the problem definition for the multi-source contact event query processing. The notations used in this paper are first shown in Table II.

Definition 1 (Trajectory): A trajectory of a moving object $o_i \in \mathcal{O}$ is a sequence of time-ordered location points, denoted as $tra_i = \{(p_1^i, t_1), (p_2^i, t_2), \ldots, (p_m^i, t_m)\}$, where p_j^i is a 2-dimensional spatial location of o_i at time point t_j. The set of time points for the trajectory is denoted as $T = \{t_1, t_2, \ldots, t_m\}$.

Definition 2 (Sliding Window): Given a window width threshold τ, a sliding window W_i is composed of τ consecutive time points starting from the time point t_i, $W_i =< t_i, t_{i+1}, \cdots, t_{i+\tau-1} >$ and $W_i \subset T$.

- $W_i[j]$ denotes the j-th time point in the sliding window W_i.
- $W_i \lessdot W_j$ indicates that the sliding window W_i appears earlier than W_j, i.e. $W_i[1] < W_j[1]$.

Definition 3 (Single-time-point Contact): Given a set of contact sources $\mathcal{S} \subset \mathcal{O}$, an innocent moving object $o_i \in \mathcal{O} - \mathcal{S}$

TABLE II: Notations

Notation	Definition
$\mathcal{O}$	The set of moving objects, $\mathcal{O} = \{o_1, o_2, \ldots, o_n\}$.
$\mathcal{S}$	The set of contact sources, $\mathcal{S} \subset \mathcal{O}$.
$\mathcal{O} - \mathcal{S}$	The set of innocent moving objects.
T	The time series of trajectory sampling, $T = \{t_1, t_2, \ldots, t_m\}$.
Tra	The trajectory dataset of moving objects, $Tra = \{tra_i \| o_i \in \mathcal{O}\}$, and tra_i is the trajectory of o_i.
τ	The width threshold of sliding window.
d	The contact distance threshold.
α	The loose threshold.
θ	The grid partition threshold.
R	The result of a MLC-event query.
$dist(o_i, o_j, t)$	The distance between o_i and o_j at time point t.

and a time point t_j, if $\exists s_x \in \mathcal{S}(dist(o_i, s_x, t_j) \leq d)$ holds, then a single-time-point contact occurs from o_i to contact sources in $\mathcal{S}$ at t_j, where $dist(o_i, s_x, t_j)$ indicates the distance between o_i and s_x at t_j and d is the distance threshold. We denote $S_{t_j}^{o_i}$ as the source set of the single-time-point contact.

$$S_{t_j}^{o_i} = \{s_x | s_x \in \mathcal{S} \wedge dist(o_i, s_x, t_j) \leq d\}. \quad (1)$$

Definition 4 (Multi-source Loose Contact Event, MLC-event): An MLC-event $e = (\mathcal{L}_i, o_i, W_j)$ occurs when a moving object o_i has $\alpha \cdot \tau$ single-time-point contacts within the earliest sliding window W_j, where $\mathcal{L}_i =< S_{W_j[1]}^{o_i}, S_{W_j[2]}^{o_i}, \cdots, S_{W_j[\tau]}^{o_i} >$ is the list of contact source sets of single-time-point contacts with o_i at every time points in W_j, $\alpha \in (0, 1]$ is the loose threshold, and τ is the width of sliding window. Formally, the MLC-event e should satisfy the following conditions:

- $|\{W_j[k] \in W_j \,|\, \exists s_x \in \mathcal{S},\ \mathrm{dist}(o_i, s_x, W_j[k]) \leq d\}| \geq \alpha \cdot \tau$
- $\forall W_l \lessdot W_j,\ |\{W_l[k] \in W_l \,|\, \exists s_x \in \mathcal{S},\ \mathrm{dist}(o_i, s_x, W_l[k]) \leq d\}| < \alpha \cdot \tau$

Definition 5 (MLC-event Query): Given a moving object set $\mathcal{O}$, a contact source set $\mathcal{S}$, a distance threshold d, a sliding window width threshold τ and a loose threshold α, an MLC-event Query $Q = (\mathcal{O}, \mathcal{S}, d, \tau, \alpha)$ is to get a result set R which contains all MLC-events caused by $\mathcal{S}$.

This paper aims to develop an MLC-event query processing framework that leverages moving-object trajectory data to accurately detect all MLC-events generated by contact sources. The baseline query processing is first proposed, and then optimized countermeasures are suggested to further improve the efficiency of search processing.

IV. The Baseline Query Processing

In this section, we propose the baseline MLC-event query processing (MLCEQ), which discovers all MLC-events using a sliding window-based analysis method. During each sliding window, every innocent moving objects are analyzed individually and checked to see whether they are involved in MLC-events. Inspired by the basic idea, we present the baseline MLC-event query processing algorithm (MLCEQ) as shown in Algorithm 1.

In Algorithm 1, a count array $count[1, 2, \cdots, n]$ is initialized to record the number of single-time-point contacts for each innocent moving object in each sliding window. Lines 5 - 14 in Algorithm 1 aim to obtain the single-time-point contacts for each innocent object within each sliding window W_k. Specifically, MLCEQ iterates through each innocent moving object o_j in $\mathcal{O} - \mathcal{S}$, calculating the distances between o_j and each contact source s_x in $\mathcal{S}$ at each time point in W_k. If the distance is smaller than the distance threshold d, a single-time-point contact occurs and the count $count[j]$ is incremented. Upon reaching the end of the sliding window, if $count[j]$ is not smaller than the preset threshold $\alpha \cdot \tau$, then an MLC-event is identified and appended to the query result, and the moving object o_j is appended to the contact source set.

Algorithm 1: MLCEQ($\mathcal{O}, \mathcal{S}, Tra, d, \tau, \alpha$)

Input : The moving object set $\mathcal{O} = \{o_1, o_2, \cdots, o_n\}$, the contact sources $\mathcal{S}$, the trajectory set Tra corresponding to $\mathcal{O}$, the distance threshold d, the sliding window width threshold τ, the loose threshold α.

Output: The set of discovered MLC-events R.

```
1  Initialize R ← ∅;
2  Initialize a count array count[1, 2, ··· , n];
3  foreach W_k ⊂ T do
4      Set count[1, 2, ··· , n] ← 0;
5      foreach t_i ∈ W_k do
6          foreach o_j ∈ O − S do
7              flag ← 0;
8              foreach s_x ∈ S do
9                  if dist(o_j, s_x, t_i) ≤ d then
10                     S_{t_i}^{o_j} ← S_{t_i}^{o_j} ∪ {s_x};
11                     if flag = 0 then
12                         count[j] ← count[j] + 1;
13                         flag ← 1;
14             L_j ← L_j ∪ {S_{t_i}^{o_j}};
15         if t_i = W_k[τ] then
16             foreach o_j ∈ O − S do
17                 if count[j] ≥ α · τ then
18                     Construct an MLC-event
                         e = (L_j, o_j, W_k);
19                     R ← R ∪ {e};
20                     S ← S ∪ {o_j};
21 return R;
```

We give an example to describe MLCEQ algorithm as shown in Fig. 2. Assuming that $\tau = 5$ and $\alpha = 0.8$, in the sliding window W_1, there are only three time points at which the distance between o_1 and the source s_1 or s_2 is greater than d, thus there are three single-time-point contacts that occur in W_1, which is smaller than $\alpha \cdot \tau = 4$, indicating no MLC-event occurs in W_1. In the sliding window W_2, there are four single-time-point contacts which is not smaller than $\alpha \cdot \tau$, thus an MLC-event $e = (< \{s_1\}, \{s_2\}, \{s_1, s_2\}, \varnothing, \{s_1\} >, o_1, W_2)$ is discovered and o_1 is added to $\mathcal{S}$ as a new contact source.

In summary, the baseline approach employs a sliding-window mechanism to compute all pairwise distances between every moving object and each contact source in order to detect all MLC-events. Upon each detection, the contact source set is refreshed, guaranteeing the discovery of all possible contact events.

V. The Optimized Query Processing

In the baseline query processing, it is necessary to calculate all the distances between each moving object and each source to determine whether the objects are involved in MLC-events.

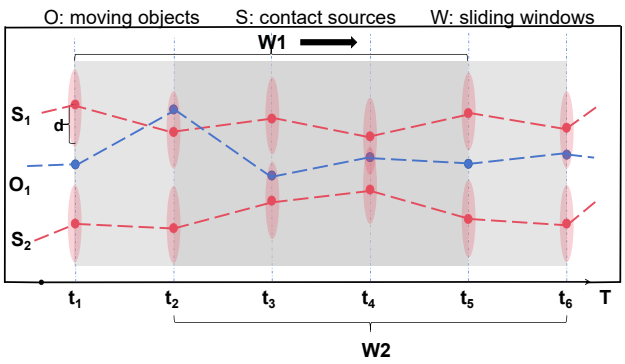


Fig. 2: An example of MLCEQ

The query speed is relatively slow and the process is time-consuming. However, many innocent moving objects which are far from the contact sources and unlikely to have single-time-point contacts still need to be checked. Detecting and filtering out such innocent objects will save time during query processing. Meanwhile, some innocent moving objects are located very close, and it's unnecessary to calculate the distances to contact sources for each of them. Based on these ideas, we introduce an InGrid-MBR inverted index optimization. We begin by introducing the structure of the InGrid-MBR-based Inverted Index, and subsequently outline the optimized MLC-event query processing method (MLCEQ+).

A. InGrid-MBR-based Inverted Index

In this section, we will introduce the InGrid-MBR-based Inverted Index to quickly filter out the innocent moving objects have no chance to have single-time-point contacts with contact sources. We begin by dividing the two-dimensional space into grid cells.

Definition 6 (Grid-based Space Representation): For a 2-dimensional spatial space Ω and a grid generation parameter θ, Ω is equally partitioned into $\theta \times \theta$ grids. Thus, the spatial space can be represented as $\Omega = \{g_1, g_2, \cdots, g_u\}$, where $u = \theta \times \theta$ and $g_i \in \Omega$ is the unique ID of a grid.

Once the space is partitioned into grids, every trajectory point is assigned to a specific grid cell. For objects that fall within the same grid, their distances to each other are relatively close. Thus, we construct the Minimum Bounding Rectangle (MBR) for the objects within the same grid. Based on grid-based space representation and the idea of MBR, we design an InGrid-MBR-based Inverted Index, which can be used to quickly retrieve innocent moving objects and their corresponding MBR.

Definition 7 (InGrid-MBR-based Inverted Index): Given a moving object dataset $\mathcal{O} = \{o_1, o_2, \cdots, o_n\}$, a time series of trajectory sampling T, and a partitioned 2-dimensional space Ω, an InGrid-MBR-based inverted index is constructed for each time point $t_j \in T$ as a set of triples $\Pi_{t_j} = \{(g_i, A_i, R_i) \mid g_i \in \Omega\}$, where g_i denotes a grid cell, A_i is the set of objects located in g_i at time t_j, and R_i is the MBR that encloses all objects in A_i. The complete index over all time points is denoted as $\Pi = \{\Pi_{t_j} \mid t_j \in T\}$.

Algorithm 2: BuildIndex($\mathcal{O}, Tra, T, \theta$)

Input : The set of moving object $\mathcal{O}$, the trajectory dataset Tra, the time series of trajectory sampling T, and the grid generation parameter θ.
Output: The InGrid-MBR-based inverted index Π.

```
Initialize the InGrid-MBR-based inverted index Π = ∅;
foreach t_k ∈ T do
    foreach p_k^j from tra_j ∈ Tra do
        Let g_u be the grid cell that contains p_k^j;
        if g_u is not in Π_{t_k} then
            Initialize object list A_u = ∅;
            Initialize MBR R_u;
            Add (g_u, A_u, R_u) to Π_{t_k};
        A_u ← A_u ∪ {o_j};
        Update R_u;
    Add Π_{t_k} to Π;
Return Π;
```

According to Definition 7, the InGrid-MBR-based inverted index supports efficient retrieval of all objects within each grid cell at a specific time point, along with their corresponding MBRs. The construction process is illustrated in Algorithm 2. As an example shown in Fig. 3, at each time point in T, a separate InGrid-MBR-based inverted index slice is built. Suppose that at time t_k, ten moving objects $\{o_1, o_2, \cdots, o_{10}\}$ are located within the partitioned 2-dimensional space consisting of grid cells $\{g_1, g_2, \cdots, g_9\}$. For grid cell g_2, o_1 and o_2 are added to A_2, and R_2 is updated to be the MBR that minimally encloses both o_1 and o_2. The same procedure is applied to the other grid cells.

In Definition 3, a single-time-point contact occurs when the distance between an innocent moving object and a contact source is less than a distance threshold d. Therefore, each contact source has a fixed contact range, which we refer to as the contact circle. Formally, the contact circle for a contact source s at time t is defined as a circle centered at the position p_t^s with radius d, denoted by $C(p_t^s, d)$. Only the innocent moving objects that fall within this circle can be involved in a single-time-point contact. Meanwhile, grids that overlap with the contact circle are identified as candidate grids. We provide the formal definition as follows.

Definition 8 (Candidate Grids): Given a partitioned 2-dimensional spatial space $\Omega = \{g_1, g_2, \cdots, g_u\}$, and a contact circle $C(p_t^s, d)$ at time t. Candidate grids of s at time t are those grids whose spatial regions either intersect with or are fully contained by the query circle denoted as $G(s, t)$:

$$G(s,t) = \{g_k | g_k \in \Omega, g_k \cap C(p_t^s, d) \neq \varnothing\} \tag{2}$$

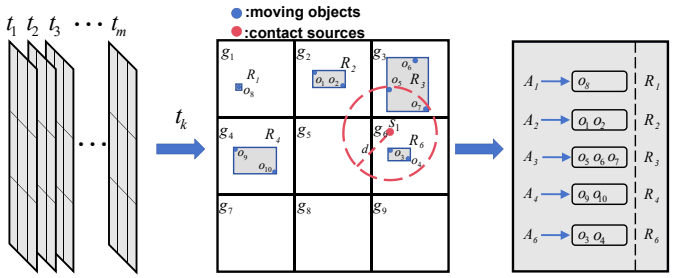


Fig. 3: An example of InGrid-MBR-based inverted index-based optimization

As shown in Fig. 3, the red circle represents the contact circle of s_1 at time t. According to Definition 8, the candidate grids of s_1 at t_k are $G(s_1, t_k) = \{g_2, g_3, g_5, g_6\}$. Once the candidate grids are identified by a contact circle, the next step is to focus on the moving objects within those grids. Since the InGrid-MBR-based inverted index allows efficient retrieval of all objects within a grid at a specific time and their corresponding MBRs, we can easily access the MBR of innocent moving objects in each candidate grid at the time. Meanwhile, innocent moving objects having no chance to have single-time-point contact with contact sources can be filtered out without redundant computations.

B. The Optimized Query Algorithm

This section introduces MLCEQ+, an optimized multi-source loose contact event query processing framework that extends the baseline MLCEQ by employing an InGrid-MBR inverted index to prune redundant distance computations.

During each sliding window, MLCEQ+ first identifies candidate grids for each contact source based on Definition 8. Then, for each candidate grid, the InGrid-MBR-based inverted index is used to retrieve both the list of objects and their MBR. By examining the spatial relationship between the MBR and the contact circle, we can determine whether a group of objects can be pruned, batch-accepted, or needs individual distance checks. Inspired by the basic idea, we present the optimized MLC-event query processing algorithm(MLCEQ+) as shown in Algorithm 3.

Algorithm 3: MLCEQ+($\mathcal{O}, \mathcal{S}, Tra, d, \tau, \alpha, \theta$)

Input : The moving object set $\mathcal{O} = \{o_1, o_2, \ldots, o_n\}$, the contact sources $\mathcal{S}$, the trajectory set Tra corresponding to $\mathcal{O}$, the distance threshold d, the sliding window width threshold τ, the loose threshold α, and the grid generation parameter θ.

Output: The set of discovered MLC-events R.

1 Initialize $R \leftarrow \varnothing$;
2 Initialize count array $count[1, 2, \ldots, n]$;
3 Partition the 2-d spatial space Ω into $\theta \times \theta$ grids;
4 $\Pi \leftarrow BuildIndex(\mathcal{O}, Tra, T, \theta)$;
5 **foreach** $W_k \subset T$ **do**
6 Set $count[1, 2, \ldots, n] \leftarrow 0$;
7 **foreach** $t_i \in W_k$ **do**
8 **foreach** $s_x \in \mathcal{S}$ **do**
9 Get the candidate girds $G(s_x, t_i)$ by $C(p_i^x, d)$;
10 **foreach** $g_u \in G(s_x, t_i)$ **do**
11 Retrieve A_u and R_u by g_u in Π_{t_i};
12 **if** $R_u \subseteq C(p_i^x, d)$ **then**
13 Each $o_j \in A_u$ trigger a single-time-point contact;
14 $S_{t_i}^{o_j} \leftarrow S_{t_i}^{o_j} \cup \{s_x\}$;
15 $count[j] \leftarrow count[j] + 1$;
16 **else if** $R_u \cap C(p_i^x, d) \neq \varnothing$ **then**
17 Compute distances for each $o_j \in A_u$ to s_x;
18 **if** *distance* $\leq d$ **then**
19 $S_{t_i}^{o_j} \leftarrow S_{t_i}^{o_j} \cup \{s_x\}$;
20 $count[j] \leftarrow count[j] + 1$;
21 $\mathcal{L}_j \leftarrow \mathcal{L}_j \cup \{S_{t_i}^{o_j}\}$;
22 **if** $t_i = W_k[\tau]$ **then**
23 **foreach** $o_j \in \mathcal{O} - \mathcal{S}$ **do**
24 **if** $count[j] \geq \alpha \cdot \tau$ **then**
25 Construct an MLC-event $e = (\mathcal{L}_j, o_j, W_k)$;
26 $R \leftarrow R \cup \{e\}$;
27 $\mathcal{S} \leftarrow \mathcal{S} \cup \{o_j\}$;
28 **return** R;

In Algorithm 3, similar to MLCEQ, a count array $count[1, 2, \cdots, n]$ is initialized to record the number of single-time-point contacts for each innocent moving object within a sliding window. Lines 8–9 describe how MLCEQ+ iterates over each contact source s_x and uses its contact circle $C(p_i^x, d)$ to identify candidate grids $G(s_x, t_i)$ based on Definition 8. For each candidate grid g_u (Lines 10–21), the InGrid-MBR-based inverted index is used to retrieve the object list A_u and the corresponding MBR R_u. Next, MLCEQ+ evaluates the spatial relationship between R_u and the contact circle. Specifically, if R_u is fully contained by $C(p_i^x, d)$ (Lines 12 –15), all objects in A_u are considered to have a single-time-point contact with s_x. In the case of partial overlap, where $R_u \cap C(p_i^x, d) \neq \varnothing$ (Lines 16 – 20), the algorithm proceeds to calculate the individual distances between each $o_j \in A_u$ and s_x. If R_u is disjoint from $C(p_i^x, d)$, all contained objects are safely pruned without further checks. When the window reaches its end, the algorithm checks whether the number of single-time-point contacts for each object o_j exceeds the threshold $\alpha \cdot \tau$. If so, an MLC-event is constructed and added to the result set R. The object o_j is also promoted to the

contact source set $\mathcal{S}$.

We further illustrate the processing using the example shown in Fig. 3. The candidate grids of s_1 at t_k are $G(s_1, t_k) = \{g_2, g_3, g_5, g_6\}$. For grid g_2, the MBR R_2 is not intersected with the contact circle $C(p_k^1, d)$, so the objects in A_2 are safely pruned. The R_3 in g_3 partial overlaps with $C(p_k^1, d)$, then individual distances between each object in A_3 and s_1 are needed to calculate to determine whether single-time-point contacts occur at time t_k. For grid g_6, the R_6 is fully contained by $C(p_k^1, d)$, all objects in A_6 are considered to have a single-time-point contact with s_1.

In conclusion, we proposed an optimized query processing MLCEQ+ utilizing the InGrid-MBR-based inverted index, which significantly improves the efficiency of multi-source loose contact event detection by reducing unnecessary calculations.

VI. Algorithm Analysis

In this section, we analyze the time complexity of the proposed algorithms, MLCEQ and MLCEQ+.

Let n be the average number of innocent objects, τ the width of each sliding window, δ the total number of sliding windows, and β the average number of contact sources at each time. For MLCEQ (Algorithm 1), at each time point within a sliding window the algorithm checks every innocent object to determine whether it has single-time point contact with each source. Thus, a per-window cost is $O(n\tau\beta)$. Then MLCEQ need to process $O(\delta)$ time windows. The time complexity of MLCEQ is shown in Eq.(3).

$$O(MLCEQ) = O(n\tau\beta) \times O(\delta) = O(\delta n\tau\beta) \tag{3}$$

Similarly in MLCEQ+, we assume that average number of candidate grids returned per source is λ and the average number of innocent moving objects per grid is η.

MLCEQ+ first finds candidate grids at each time point within a sliding window and check the MBR relationship between the query circle of the source and the objects within the grid. The time complexity of this phase is $O(\lambda\tau\beta)$. Then, a portion of grids (proportional to μ) among the λ grids are further processed to compute exact distances for the η objects they contain. The time complexity is $O(\mu\lambda\eta\tau\beta)$.

Thus, the total time complexity of MLCEQ+ is shown on Eq.(4).

$$\begin{aligned} O(MLCEQ+) &= (O(\lambda\tau\beta) + O(\mu\lambda\eta\tau\beta)) \times O(\delta) \\ &= O\left(\delta\lambda\tau\beta(1+\mu\eta)\right) \end{aligned} \tag{4}$$

Combining Eqs. (3) and (4), and noting that $\lambda(1+\mu\eta) \ll n$, it follows that MLCEQ+ can markedly accelerate the query process. From the above analysis, it can be seen that our proposed optimized algorithm ensures scalability for large-scale datasets by minimizing unnecessary computations while maintaining detection accuracy. In the subsequent experiments, we further evaluate the performance of the proposed algorithms.

VII. Performance Evaluation

In this section, we comprehensively evaluate the proposed MLCEQ and MLCEQ+ algorithms using two real-world trajectory datasets, Taxi [21] and TDrive [22]. They contain 4315 and 4142 trajectories with 1440 and 2017 locations, respectively.

Our experiments were conducted on an Intel i7-12700KF CPU with 32 GB of RAM, running 64-bit Windows 11 and Java development tools. Table III details the default parameter settings for the T-Drive and Taxi datasets.

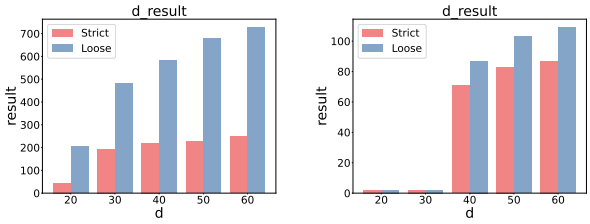
TABLE III: Default parameter settings for T-drive and Taxi

Notations	Meanings	Taxi	TDrive
n	the number of moving objects	4315	4142
m	the number of time points	1440	2017
d	the distance threshold	20	50
τ	the sliding window width threshold	12	12
α	the loose threshold	0.8	0.8

A. Query Result Evaluation

In this subsection, we evaluate the query results under varying parameters, comparing multi-source strict(traditional definition, where the loose threshold is set to $\alpha = 1.0$) and loose contact event queries. For simplicity, we refer to these as "strict mode" and "loose mode", respectively. The results are presented in Fig. 4–7.

Fig. 4 shows how the query results vary with the distance threshold d for both the Taxi and T-Drive datasets. Overall, the number of results increases with larger d values in both datasets. Due to the sparser spatial distribution in T-Drive, the query result remains relatively low when d is small (20–30). In all cases, loose mode consistently returns more results than strict mode, and the results from strict mode are fully contained within those from loose mode.

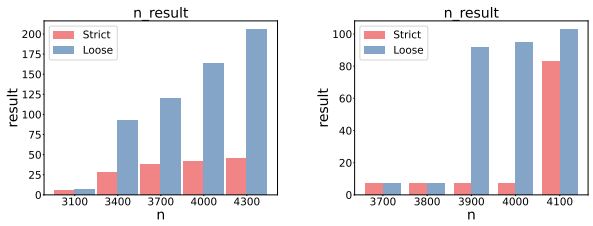


(a) Taxi (b) T-Drive

Fig. 4: Query result versus d.

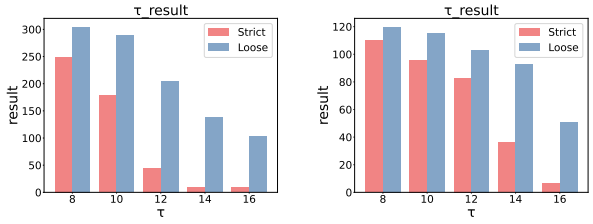
Fig. 5 demonstrates that query results increase with the number of innocent objects n, and the same containment relationship between loose and strict modes holds.

Fig. 6 illustrates the effect of the sliding window width τ on query results. As τ increases from 8 to 16, the number of results decreases in both datasets. This is because $\alpha \cdot \tau$ increases with larger τ, making it harder for objects to satisfy the condition. Again, loose mode yields more results than strict mode across all values of τ.



(a) Taxi (b) T-Drive

Fig. 5: Query result versus n.



(a) Taxi (b) T-Drive

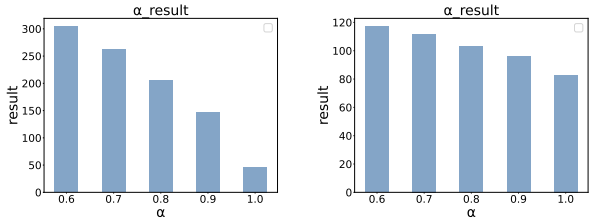
Fig. 6: Query result versus τ.

Finally, Fig. 7 shows how varying the loose threshold α influences the query results. As α increases from 0.6 to 0.9 (loose mode), the number of results gradually declines, yet remains consistently higher than those under the strict mode ($\alpha = 1.0$).

Our experimental evaluation investigates the impact of four critical parameters on the results of contact event query, showing that loose contact queries consistently return more results than strict ones, and all strict results are a subset of those found by loose queries. These additional events are not false positives; they represent realistic scenarios where intermittent but frequent proximity may still pose transmission risks. Identifying such contacts is crucial in public health contexts, where missed exposures can lead to secondary transmission and undermine containment efforts.

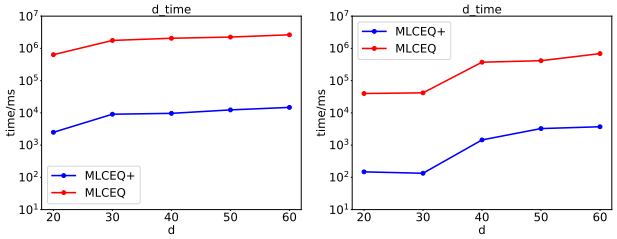
B. Query Time Evaluation

In this subsection, we compare the execution times of the baseline MLCEQ and the optimized MLCEQ+ under varying parameter settings. The findings are illustrated in Fig. 8 through Fig. 11.



(a) Taxi (b) T-Drive

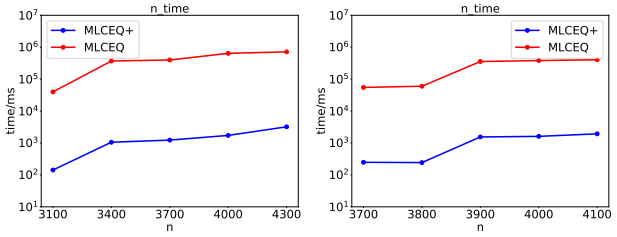
Fig. 7: Query result versus α.



(a) Taxi (b) T-Drive

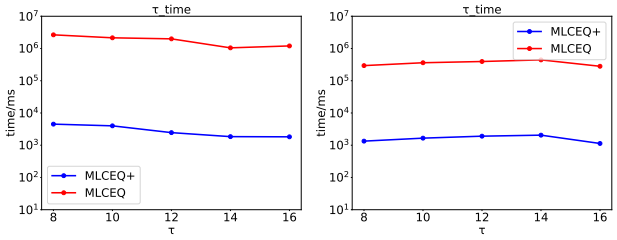
Fig. 8: Query time versus d.

Fig. 8 shows MLCEQ+ significantly outperforms MLCEQ in query time under varying distance thresholds d. Both algorithms show increasing time costs with larger d, aligning with the growth in the total number of contact results in Fig. 4. This correlation arises because larger d leads to more innocent moving objects becoming sources and thereby require both algorithms to process a larger subset of sources.



(a) Taxi (b) T-Drive

Fig. 9: Query time versus n.



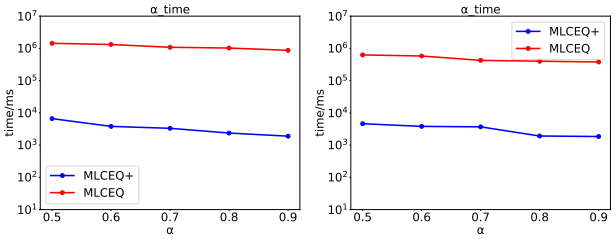
(a) Taxi (b) T-Drive

Fig. 10: Query time versus τ.

As shown in Fig. 9, in both dataset, the query time of MLCEQ and MLCEQ+ increase with the number of innocent moving objects n.

The Fig. 10 and 11 illustrate the query time performance for the Taxi (a) and T-Drive (b) datasets as the sliding window width τ and the loose threshold α increases. Both figures reveal a consistent downward trend in query time, which similarly correlates with the decrease in the total number of contact events detected.

In summary, across all tested parameter settings, MLCEQ+ reliably identifies multi-source loose contact events and consistently outperforms MLCEQ in query efficiency.



(a) Taxi (b) T-Drive

Fig. 11: Query time versus α.

VIII. Conclusion

This paper addresses the limitations of traditional contact tracing by proposing a multi-source loose contact query framework. We first formalize the definition of loose contact events and introduce a baseline algorithm (MLCEQ) using the sliding window mechanism for event detection. To improve efficiency, we further propose MLCEQ+, which leverages an InGrid-MBR-based inverted index. Experiments show that our approach detects more contact events than traditional strict methods, while the optimized version significantly enhances query performance. Identifying such intermittent contacts can reduce undetected transmission risk. In future work, we plan to incorporate privacy-preserving mechanisms into the query framework to address the growing concerns over personal trajectory data security in contact tracing applications.

References

[1] J. Bao, Y. Zheng, and M. F. Mokbel, "Location-based and preference-aware recommendation using sparse geo-social networking data," in *SIGSPATIAL 2012 International Conference on Advances in Geographic Information Systems, SIGSPATIAL'12, Redondo Beach, CA, USA, November 7-9, 2012*, I. F. Cruz, C. A. Knoblock, P. Kröger, E. Tanin, and P. Widmayer, Eds. ACM, 2012, pp. 199–208.

[2] J. Dai, B. Yang, C. Guo, and Z. Ding, "Personalized route recommendation using big trajectory data," in *2015 IEEE 31st International Conference on Data Engineering*, IEEE. IEEE, 2015, pp. 543–554.

[3] F. Ahmed and Y. E. Hawas, "An integrated real-time traffic signal system for transit signal priority, incident detection and congestion management," *Transportation Research Part C: Emerging Technologies*, vol. 60, pp. 52–76, 2015.

[4] T. Morita, A. Toyoda, S. Aisu, A. Kaneko, N. Suda-Hashimoto, I. Adachi, I. Matsuda, and H. Koda, "Animals exhibit consistent individual differences in their movement: A case study on location trajectories of japanese macaques," *Ecological Informatics*, vol. 56, pp. 101 057–101 057, 2020.

[5] Y. Zheng, "Urban computing: enabling urban intelligence with big data," *Frontiers of Computer Science*, vol. 11, no. 1, pp. 1–3, 2017.

[6] H. R. Fuller, K. J. Ajrouch, and T. C. Antonucci, "The convoy model and later-life family relationships," *Journal of Family Theory & Review*, vol. 12, no. 2, pp. 126–146, 2020.

[7] L. Tang, Y. Zheng, J. Yuan, J. Han, A. Leung, C. Hung, and W. Peng, "On discovery of traveling companions from streaming trajectories," in *Proceedings of the 28th IEEE International Conference on Data Engineering*. IEEE, 2012, pp. 186–197.

[8] D. Zhang, Y. Zhang, and C. Zhang, "Data mining approach for automatic ship-route design for coastal seas using AIS trajectory clustering analysis," *Ocean Engineering*, vol. 236, pp. 109 535–109 535, 2021.

[9] R. Shibasaki. (2017) Call detail record (cdr) analysis: Sierra leone.

[10] H. Stevens and M. B. Haines, "Tracetogether: Pandemic response, democracy, and technology," *East Asian Science, Technology and Society: An International Journal*, vol. 14, no. 3, pp. 523–532, 2020.

[11] J. Hendry, "Australia's COVID tracing app better than Singapore's: Health chief," ITNews, Apr 2020, accessed: 2024-10-24.

[12] Q. Lin and J. Son, "A close contact tracing method based on bluetooth signals applicable to ship environments," *KSII Transactions on Internet and Information Systems (TIIS)*, vol. 17, no. 2, pp. 644–662, 2023.

[13] S. Hisada, T. Murayama, K. Tsubouchi, S. Fujita, S. Yada, S. Wakamiya, and E. Aramaki, "Surveillance of early stage COVID-19 clusters using search query logs and mobile device-based location information," *Scientific Reports*, vol. 10, no. 1, pp. 18 680–18 680, 2020.

[14] A. B. Dar, A. H. Lone, S. Zahoor, A. A. Khan, and R. Naaz, "Applicability of mobile contact tracing in fighting pandemic (COVID-19): Issues, challenges and solutions," *Computer Science Review*, vol. 38, pp. 100 307–100 307, 2020.

[15] L. Reichert, S. Brack, and B. Scheuermann, "A survey of automatic contact tracing approaches using bluetooth low energy," *ACM Transactions on Computing for Healthcare*, vol. 2, no. 2, pp. 1–33, 2021.

[16] J. Xu, H. Lu, and Z. Bao, "IMO: A toolbox for simulating and querying "infected" moving objects," *Proc. VLDB Endow.*, vol. 13, no. 12, pp. 2825–2828, 2020.

[17] L. Alarabi, S. Basalamah, A. Hendawi, and M. Abdalla, "Traceall: A real-time processing for contact tracing using indoor trajectories," *Information*, vol. 12, no. 5, pp. 202–202, 2021.

[18] Y. Chen, H. Dai, G. Yang, and Y. Chen, "Multi-source infection pattern mining algorithms over moving objects," in *2022 IEEE 25th International Conference on Computer Supported Cooperative Work in Design (CSCWD)*. IEEE, 2022, pp. 1102–1107.

[19] P. Li, H. Dai, Y. Chen, B. Li, and G. Yang, "Efficient multi-source contact event query processing for moving objects," in *IEEE International Conference on Data Mining, ICDM 2023, Shanghai, China, December 1-4, 2023*, G. Chen, L. Khan, X. Gao, M. Qiu, W. Pedrycz, and X. Wu, Eds. IEEE, 2023, pp. 1109–1114.

[20] P. Li, H. Dai, Q. Zhou, Y. Chen, Q. Zhou, B. Li, and G. Yang, "ECEQ: efficient multi-source contact event query processing for moving objects," *World Wide Web (WWW)*, vol. 27, no. 6, pp. 69–69, 2024.

[21] S. C. R. Group, "Taxi," https://www.cse.ust.hk/scrg/, 2010.

[22] J. Yuan, Y. Zheng, X. Xie, and G. Sun, "Driving with knowledge from the physical world," in *Proceedings of the 17th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2011, pp. 316–324.